

COSI 180: Algorithms

Lecture 18: Exact Cover

Christopher

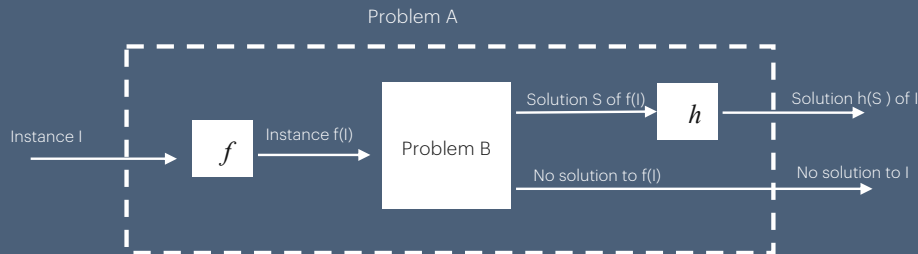
1

Basic Reduction Strategies

- Reduction by simple equivalence
- Reduction from special case to general case
- Reduction by encoding with gadgets

2

Implications of poly-time reductions



- If $A \leq_p B$ and $B \in P$, then $A \in P$ Design algorithms.
- If $A \leq_p B$ and $A \notin P$, then $B \notin P$ Establish intractability
- If both $A \leq_p B$ and $B \leq_p A$, we use notation $A \equiv_p B$. In this case, A can be solved in polynomial time iff B can be. Establish equivalence.

3

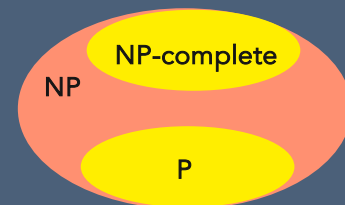
NP-Complete

- A problem B is **NP-complete** if:

(1) $B \in \text{NP}$

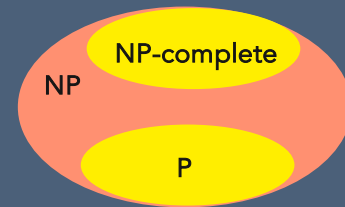
(2) $A \leq_p B$ for all $A \in \text{NP}$

- If B satisfies only property (2) we say that B is **NP-hard**
- No polynomial time algorithm has been discovered for an **NP-Complete** problem
- No one has ever proven that no polynomial time algorithm can exist for any **NP-Complete** problem



4

NP-Complete



- A problem B is **NP-complete** if:
 - (1) $B \in \text{NP}$
 - (2) $A \leq_p B$ for some $A \in \text{NP-Complete}$ (typically $A = 3\text{SAT}$)
- If B satisfies only property (2) we say that B is **NP-hard**
- No polynomial time algorithm has been discovered for an **NP-Complete** problem
- No one has ever proven that no polynomial time algorithm can exist for any **NP-Complete** problem

5

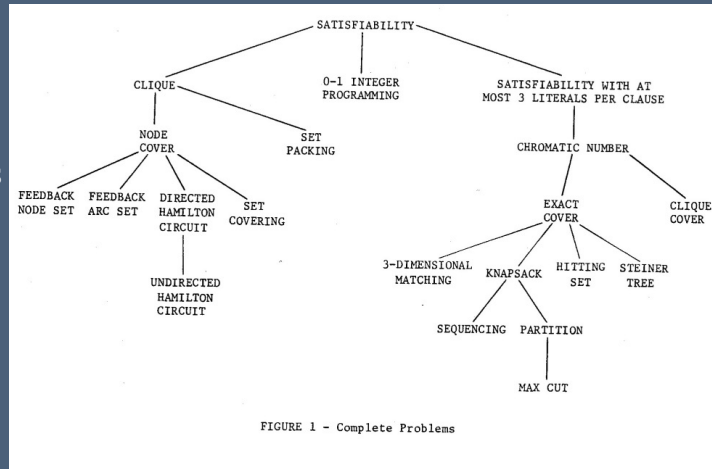
Cook-Levin Theorem

- Proved Boolean satisfiability problem is NP-complete
- Details are complicated (requires a formal definition of computation i.e. Turing Machines)
- High Level Idea:
 - Every problem in NP has a non-deterministic algorithm that runs in polynomial time
 - If the algorithm accepts an input, there is a series of steps in polynomial time that it takes to reach that conclusion
 - The initial input, and steps can be represented as a Boolean expression
 - The Boolean expression is satisfiable if and only if the algorithm outputs true for given input

6

Karp's 21 NP-complete problems

- After proving SAT is NP-complete
- Showing other problems are NP-complete via reduction is as simple as choosing favorite NP-complete problem and reducing to other problem



7

Exact Cover problem

Given a matrix of 0s and 1s, does it have a set of rows containing exactly one 1 in each column?

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

8

Exact Cover problem

Given a finite nonempty set U (the universe), and a set F of subsets of U , is there an exact cover $C \subseteq F$ such that the sets in C are disjoint and their union is equal to U ?

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

$U = \{a, b, c, d, e, f, g\}$

$F = \{\{c, e\},$
 $\{a, d, g\},$
 $\{b, c, f\},$
 $\{a, d, f\},$
 $\{b, g\},$
 $\{d, e, g\}\}$

$C = \{\{c, e\}, \{a, d, f\}, \{b, g\}\}$

9

Exact Cover is in NP

- Proof (verifier)
- Let V be an algorithm that takes as input U , F , and C
 - Verify that every set in F is a subset of U
 - Verify that every set in C is an element in F
 - Verify that every element of U belongs to exactly one element of C

10

Exact Cover is NP-hard

- Proof (via reduction)
- We will show that $3\text{-SAT} \leq_p \text{Exact Cover}$
- Given an instance of 3-SAT, denoted as a set $S = \{C_1, C_2, \dots, C_m\}$ from n Boolean variables $\{x_1, x_2, \dots, x_n\}$, we must construct an instance $f(S) = (U, F)$ of Exact Cover such that S is satisfiable if and only if $f(S)$ has an exact cover
- Idea:
 - Build a gadget for every clause to be true if at least one of its literals is true
 - Build a gadget for every propositional variable to assign it either true or false
 - Build a gadget to take care of any remaining literals that weren't used

11

Exact Cover is NP-hard (gadget 1)

- $S = \{C_1, C_2, \dots, C_m\}$ from n Boolean variables $\{x_1, x_2, \dots, x_n\}$
- Denote $C_j = (L_{j1} \vee L_{j2} \vee L_{j3})$
- Ensure every clause is satisfied
- For every clause C_j , let $\{C_j, p_{j1}\}$, $\{C_j, p_{j2}\}$, and $\{C_j, p_{j3}\}$ be in F
- The idea is that as long as one of the literals p_{jk} can evaluate to true, C_j can be covered

12

Exact Cover is NP-hard (gadget 2)

- $S = \{C_1, C_2, \dots, C_m\}$ from n Boolean variables $\{x_1, x_2, \dots, x_n\}$
- Denote $C_i = (L_{i1} \vee L_{i2} \vee L_{i3})$
- Assigning values to variables
- Let $X_{i, \text{true}} = \{x_i\} \cup \{p_{jk} \mid L_{jk} = \bar{x}_i\}$
- Let $X_{i, \text{false}} = \{x_i\} \cup \{p_{jk} \mid L_{jk} = x_i\}$
- Add $X_{i, \text{true}}$ and $X_{i, \text{false}}$ to F
- The idea is that if x_i is true, then it shouldn't be possible for any clause to be covered using the opposite assignment

13

Exact Cover is NP-hard (gadget 3)

- $S = \{C_1, C_2, \dots, C_m\}$ from n Boolean variables $\{x_1, x_2, \dots, x_n\}$
- Denote $C_i = (L_{i1} \vee L_{i2} \vee L_{i3})$
- Dealing with potential uncovered variables
- For every p_{jk} add $\{p_{jk}\}$ to F
- After assigning every variable to true or false, and after satisfying all Clauses, any remaining p_{jk} items should be covered

14

Exact Cover is NP-hard

- Let $f(S) = (U, F)$ where F is described from the last three slides and $U = \{x_i\} \cup \{C_j\} \cup \{p_{jk}\}$
- If v is a valid assignment of variables that satisfies S , then we can make an exact cover C as follows:
 - For each x_i , we put $X_{i, \text{true}}$ in C if $v(x_i) = T$, else we put $X_{i, \text{false}}$ in C
 - For each C_j , we put some subset $\{C_j, p_{jk}\}$ in C for a literal L_{jk} which is made true by v .
 - By construction, of $X_{i, \text{true}}$ and $X_{i, \text{false}}$, this p_{jk} is not in any set in C selected so far. In particular because v is a valid assignment, such a literal exists for every clause
 - Having covered all x_i and C_j , we put a set $\{p_{jk}\}$ in C for every remaining p_{jk} which has not been covered by the sets already in C
- Conversely, if C is an exact cover of $f(S)$, then we can define a truth assignment by setting x_i to true if $X_{i, \text{true}}$ is in C else setting x_i to be false

15

Solving Exact Cover

Given a matrix of 0s and 1s, does it have a set of rows containing exactly one 1 in each column?

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

Naïve algorithm:

hasExactCover(M)

- For every subset of rows in M
 - Check if rows are non-conflicting
 - If rows cover every column in M
 - Return True
- Return False

16

Solving Exact Cover

Given a matrix of 0s and 1s, does it have a set of rows containing exactly one 1 in each column?

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

Faster algorithm:

hasExactCover(M)

- If M has no columns return True
- Choose a column c of M
- For every row where c is set to 1
 - Reduce M to be compatible with row
 - If hasExactCover(reduced M) returns True
 - Return True
- Restore M
- Return False

17

Exact Cover problem

Given a finite nonempty set U (the universe), and a set F of subsets of U, is there an exact cover $C \subseteq F$ such that the sets in C are disjoint and their union is equal to U?

U = {a, b, c, d, e, f, g}

F = {{c, e},
{a, d, g},
{b, c, f},
{a, d, f},
{b, g},
{d, e, g}}

C = {{c, e}, {a, d, f}, {b, g}}

Better Algorithm v2:

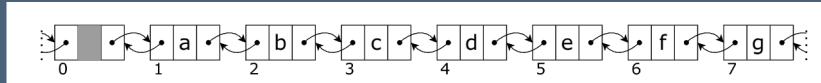
hasExactCover(U, F)

- If U is empty return True
- Select an item i in U
- For every option in F that contains i
 - Reduce U, F to be compatible with option
 - If hasExactCover(reduced U, reduced F)
 - Return True
- Restore U, F
- Return False

18

How to reduce and restore fast?

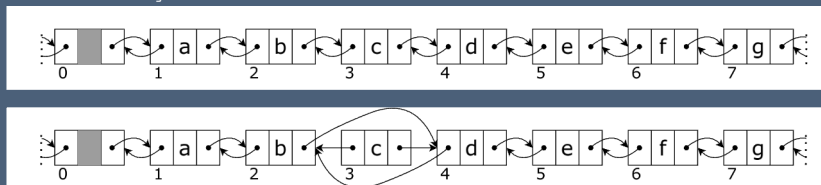
Answer: doubly linked lists



19

How to reduce and restore fast?

Answer: doubly linked lists

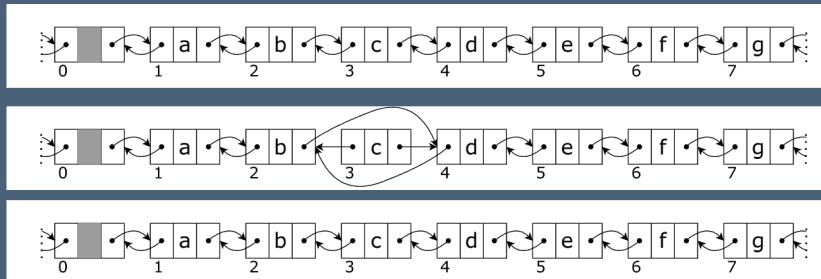


```
Delete (node)
Set r ← node.right
Set l ← node.left
r.Left = l
l.Right = r
```

20

How to reduce and restore fast?

Answer: doubly linked lists

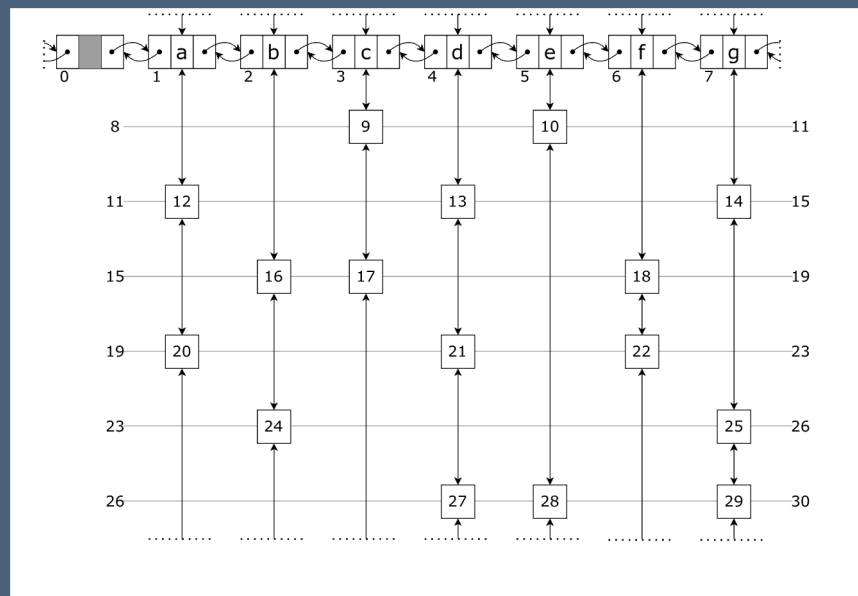


Delete (node)
Set $r \leftarrow \text{node.right}$
Set $l \leftarrow \text{node.left}$
 $r.\text{Left} = l$
 $l.\text{Right} = r$

Undelete (node)
Set $r \leftarrow \text{node.right}$
Set $l \leftarrow \text{node.left}$
 $r.\text{Left} = \text{node}$
 $l.\text{Right} = \text{node}$

21

Representation of U and F in memory



22

Algorithm X (simplified)

hasExactCover(U, F, level)

- If U is empty return True
- Select an item i in U
- Cover(i)
- For every option in F that contains i
 - For every item in option j $\neq i$
 - Cover(j)
- If hasExactCover(U, F, level + 1)
- Return True
- For every item in option j $\neq i$
- Uncover(j)
- Uncover(i)
- Return False

23

Cover(item)

Cover(item-node)

- Set $p \leftarrow \text{item-node.down}$
- While $p \neq \text{item-node}$
- Hide(p)
- $P = p.\text{down}$
- Delete(item-node)

Cover gets rid of the item from U
and any options in F that have the item

Hide gets rid of the option associated
with node p

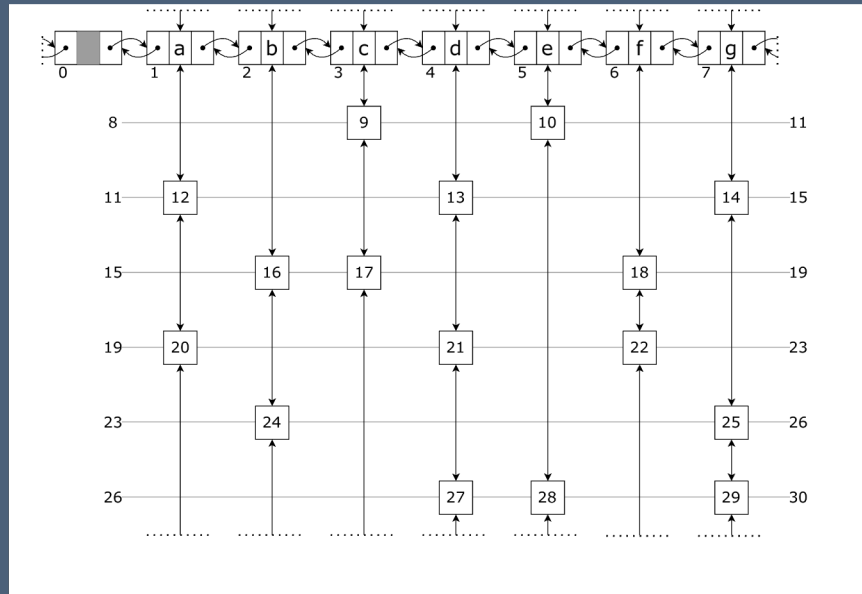
hide(p)

- Set $q \leftarrow (p+1)$
- While $q \neq p$
- If q is a spacer node
- Set $q = q.\text{up}$
- else
- $q.\text{up.down} = q.\text{down}$
- $q.\text{down.up} = q.\text{up}$
- $q.\text{top.len} \leftarrow q.\text{top.len} - 1$
- $q \leftarrow q+1$

24

Dancing links Example

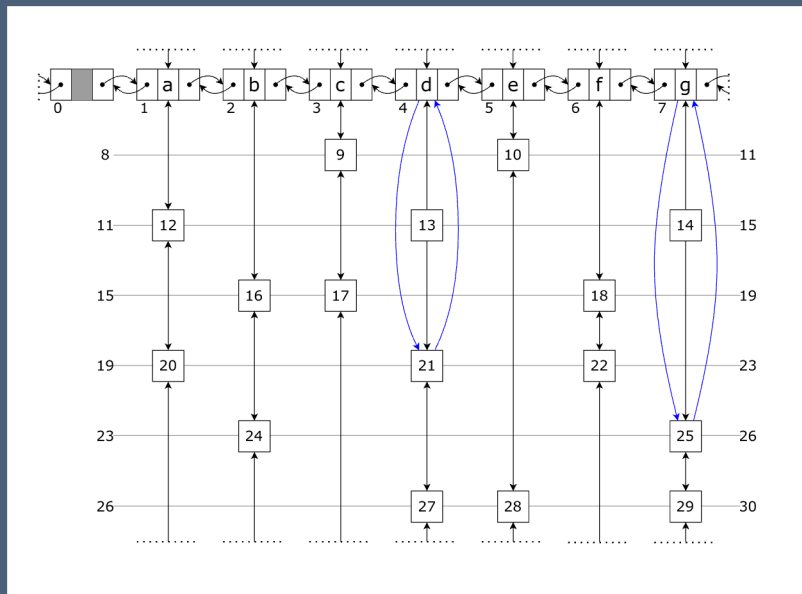
Choose item a
Cover(a)
Hide(12)
Hide(20)
Delete(1)



25

Dancing Links

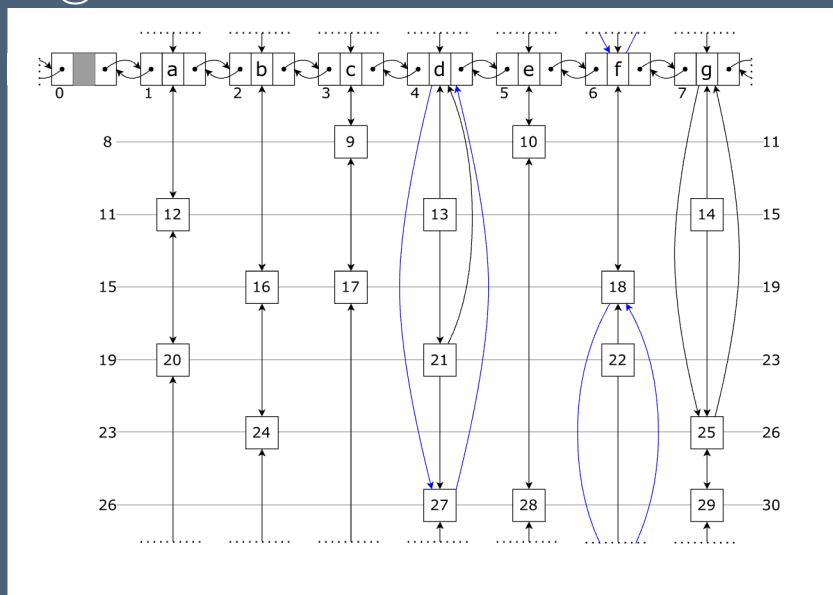
Choose item a
Cover(a)
Hide(12)
Hide(20)
Delete(1)



26

Dancing Links

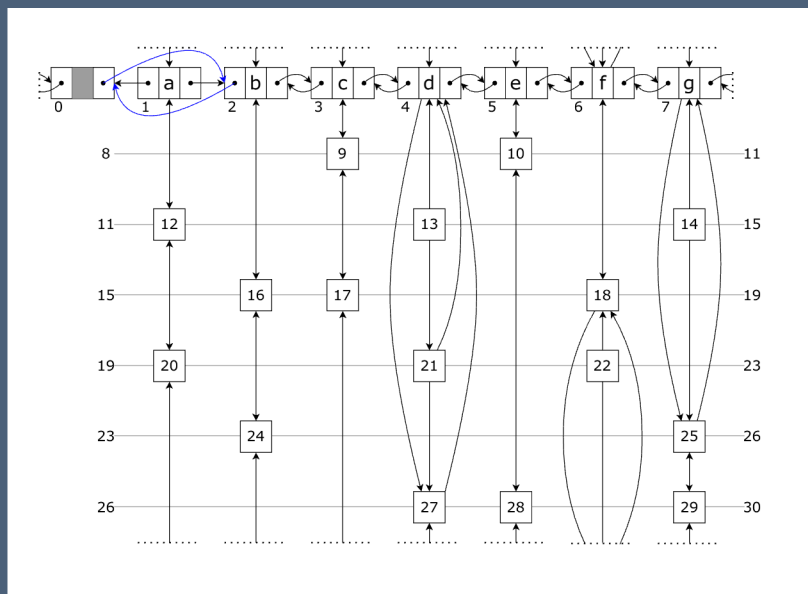
Choose item a
 Cover(a)
 Hide(12)
 Hide(20)
 Delete(1)



27

Dancing Links

Choose item a
 Cover(a)
 Hide(12)
 Hide(20)
 Delete(1)



28

Dancing links

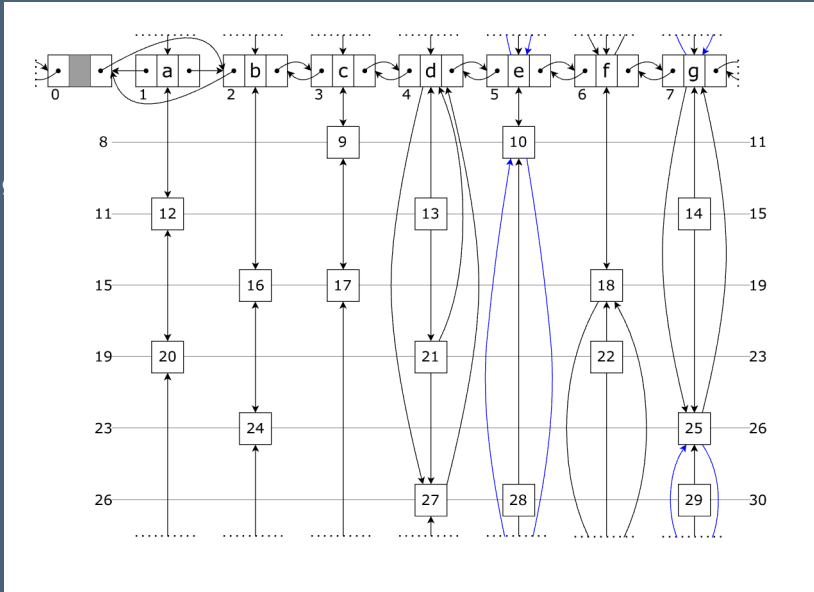
Choose option {a d e g}

Cover(a)

Cover(d)

Hide(27)

delete(4)



29

Dancing links

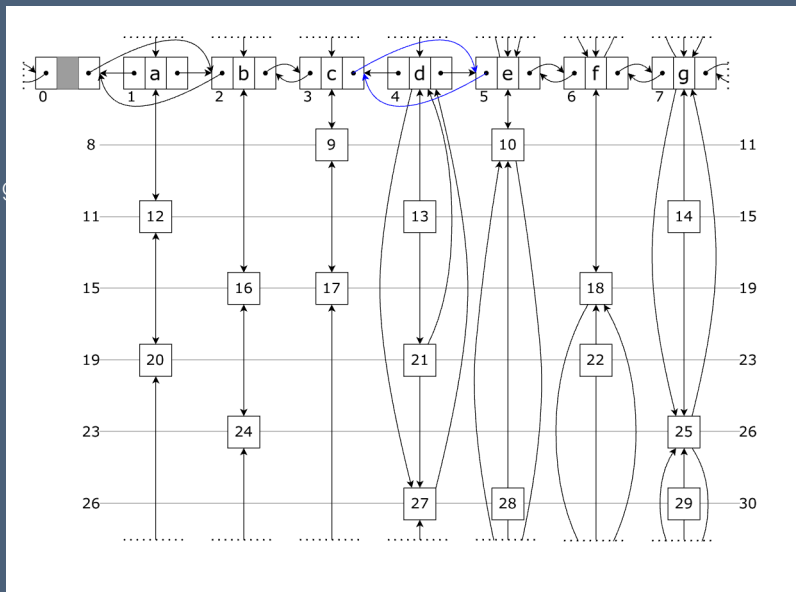
Choose option {a d e g}

Cover(a)

Cover(d)

Hide(27)

delete(4)



30

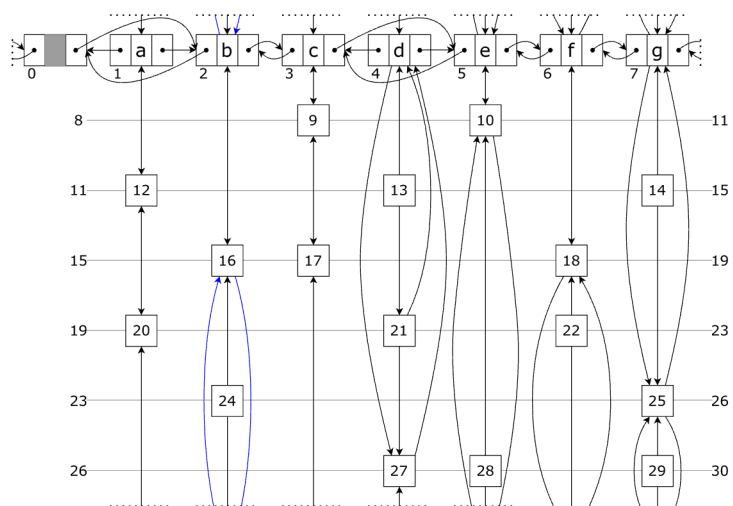
Dancing links

Choose option {a d

Cover(g)

Hide(25)

```
delete(7)
```



31

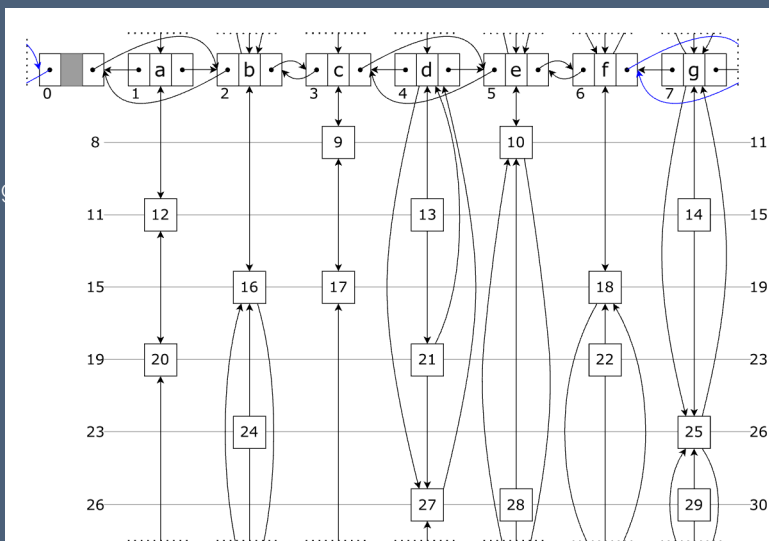
Dancing links

Choose option {a d g}

Cover(g)

Hide(25)

delete(7)



32